

OpenCollab SWE-bench 委员会 V3.1 设计与评测 Harness 报告

Kai / Codex

2026-07-06

1 问题背景

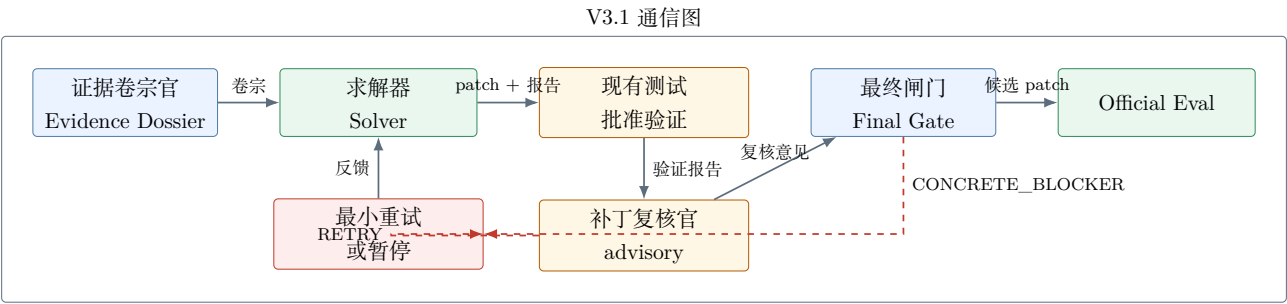
过去三天的 V2/V3 实验暴露出一个很清楚的现象：完整委员会在理念上能提供更多审查视角，但结构化 agent 数量过多时，任一环节的 transport failure 都会把整题拖入 `infra_invalid`。这些失败集中在 `candidate-contracts`、`post-validation-factory`、`post-validation-judge-triage`、`final-verify` 等阶段，summary 里 `rate_limit_error_count=0`、`schema_failure_count=0`，说明主要来源是 `anthropic.APIConnectionError: Connection error` 一类传输失败。远端代理 `/healthz` 正常只说明代理端口存活，不能保证每次上游结构化调用都成功。

另一个问题来自恢复语义。checkpoint 保存了 workflow 阶段，但旧实现遇到空 diff 时会删除之前的 worktree sidecar。这样一来，续跑时 workflow 以为可以从后段继续，容器文件系统却回到干净状态，最终日志出现 `workflow: tokens=0 steps=0, staged diff empty, git status clean`，并被写成 `empty_patch_invalid`。这种结果会把已经花掉的时间和 token 变成不可恢复成本。

V3.1 的目标是降低结构化通信失败面，把契约裁决后置到有证据的阶段，并让所有可等待恢复的问题进入暂停或 `advisory gap` 状态。它仍然保留 V3 的核心想法：先读任务、盘点语义、复用仓库 helper、审查 diff、再交给 official eval；改变的是通信协议和错误语义。

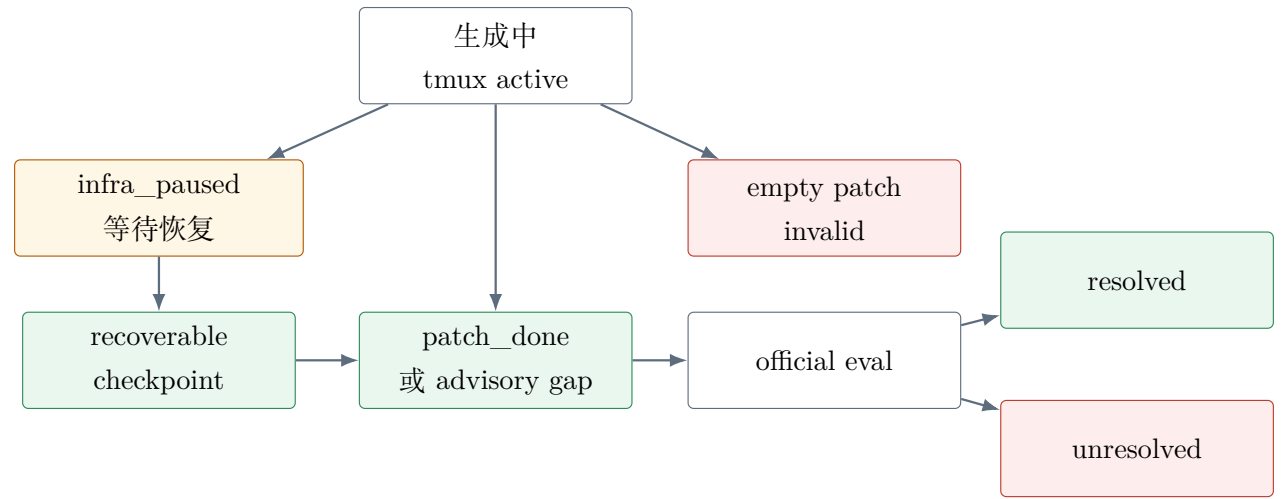
2 V3.1 总体设计

V3.1 将前半段多个硬结构化阶段合并成一个证据卷宗。这个卷宗同时包含任务接入、语义盘点、helper 审查、候选契约和探针计划。求解器拿到卷宗后直接写 patch。补丁完成后，现有测试与批准验证官给出公开验证报告；补丁复核官把 diff 风险、后置验证和最终怀疑者合并为一份 `advisory` 复核；最终闸门只判断当前 patch 是否适合送 official eval。若 `advisory` 阶段断连，workflow 记录 `done_with_advisory_gap` 和 `advisory_transport_error_count`，patch 默认保留为 `diagnostic_only` 候选。



图中的蓝色节点是结构化证据节点，绿色节点产生或消费真实 patch，黄色节点提供验证与 advisory 复核，红色节点代表需要修改代码或暂停等待的路径。V3.1 里只有求解器失败、final gate 给出具体阻断、official eval 判定 unresolved 会影响语义结果；advisory 结构化阶段的传输错误会变成可见缺口。

3 错误与恢复状态图



恢复状态的关键改动在 WorkflowContext: 空 diff 捕获时，若已经存在非空 checkpoint.worktree.patch, metadata 会写成 empty_worktree_preserved_previous_patch, 并保留旧 patch。watchdog 看到 worktree_patch_bytes>0 时把任务归入 recoverable_checkpoint。这使暂停、修复代理、恢复容器成为无损动作。

4 通信协议

消息	发送方到接收方	关键字段
----	---------	------

EvidenceDossier	证据卷宗官到求解器、验证官、复核官、最终闸门	task_frame 记录问题目标、复现线索、相关路径、缺失信息； semantic_inventory 记录源码事实、证据路径、公开测试覆盖、兼容边界、未知项；helper_audit 记录可复用 helper、调用限制、复用优先级； contract_hypotheses 区分硬契约、有证据假设、未知项；probe_plan 只描述计划；communication_notes 记录对后续 agent 的提醒。
SolverReport	求解器到验证官与复核官	修改摘要、涉及文件、复用 helper 的理由、未复用 helper 的理由、准备运行的验证命令、已知风险。
ExistingTestsVerdict	现有测试与批准验证官到复核官与最终闸门	status、findings、checks、allowed_patch_paths、disallowed_patch_paths。只有实际运行或源码检查形成的证据能写成失败。
PatchReview	补丁复核官到最终闸门与求解器	diff_risks、post_validation、skeptic、decision、retry_feedback、advisory_notes。连接失败时写 advisory_fallback=true。
FinalGateVerdict	最终闸门到 watchdog 与 official eval driver	verdict、findings、allowed_patch_paths、disallowed_patch_paths、retry_feedback、verification_coverage、done_with_advisory_gap。
CheckpointSidecar	Workflow runtime 到 watchdog 与恢复脚本	checkpoint.worktree.patch、checkpoint.worktree.json、patch_bytes、patch_sha256、capture_status、preserved_previous_patch、submission_eligible。
OfficialEvalResult	official eval driver 到 summary	record_id、patch sha、resolved、unresolved、失败摘要、report 路径。

5 Agent Prompt 详稿

5.1 证据卷宗官

你是 V3.1 证据卷宗官。你的职责是生成可供求解器使用的证据卷宗，避免把任务拆成许多硬结构化关卡。

请输出六块内容。第一块是任务框架，包含问题目标、复现线索、相关路径、缺失信息和下一步阅读文件。第二块是语义盘点，包含源码事实、证据路径、公开测试覆盖、兼容边界、未知项和风险类别。第三块是原生工具审查，列出仓库内可复用 helper、调用限制、复用优先级和手写替代风险。第四块是候选契约，把硬契约、有证据假设和未知项分开。硬契约只能来自源码、公开测试、文档或稳定 API。第五块是探针计划，只描述建议探针，不能把未运行探针写成证据。第六块是通信备注，说明哪些信息可以约束求解器，哪些信息只适合作为提醒。

这一步只做证据卷宗，不写补丁，不裁决最终通过。输出必须符合 `V31\EVIDENCE\DOSSIER\SCHEMA`，字段包括 `task_frame`、`semantic_inventory`、`helper_audit`、`contract_hypotheses`、`probe_plan`、`communication_notes`。

这个 agent 解决的是 V3 前半段“先局部裁决再理解代码”的问题。它只负责收集和分层证据，不能把契约假设升级成最终裁决。若它发生 `transport failure`，`workflow` 会生成 `fallback dossier`，并要求求解器自行补读关键文件。

5.2 求解器

你是 V3.1 求解器。请根据证据卷宗做最小源码修改。

执行规则如下。优先复用证据卷宗里列出的仓库原生 helper。未运行的探针只能当作验证计划，不能当作已经通过的证据。若证据卷宗来自 `infra fallback`，要先读取关键文件补足事实，再修改代码。修改范围只限源码和必要配置，不写临时测试文件到最终 `diff`。输出修改摘要、涉及文件、复用或手写理由、准备运行的验证命令。

输入包含任务、证据卷宗、已有验证和上一轮反馈。若上一轮 `final gate` 或 `patch review` 给出具体阻断，先修那个阻断；若只有 `advisory` 缺口，优先补足公开验证证据。

求解器是 V3.1 里最重要的硬执行节点。它的 `transport failure` 仍会暂停或进入 `infra` 状态，因为没有 `solver` 输出时无法构成 `patch`。它可以使用读写工具和 `apply_patch`，其他结构化审查节点没有写工具。

5.3 现有测试与批准验证官

你是 V3.1 现有测试与批准验证官。请只使用公开仓库、公开测试、证据卷宗批准的探针计划和当前 `diff` 做验证。

请输出结构化验证报告。字段 `status` 只能是 `PASS`、`FAIL_WITH_EVIDENCE` 或 `NOT_RUN`。字段 `checks` 写实际运行或实际检查过的命令、文件、结果和证据。字段 `allowed_patch_paths` 写当前问题允许修改的源码路径。字段 `disallowed_patch_paths` 写临时验证文件、越界文件或测试污染。

只有真实运行失败、源码检查失败或路径越界可以触发 `FAIL_WITH_EVIDENCE`。未运行探针只能写进 `findings`，不能当成失败证据。

这个节点把“验证计划”和“验证事实”分开。它支持 final gate 判断 patch 边界，也为 official eval 前的诊断提供证据。若该阶段 transport failure，V3.1 会写成 advisory_fallback，并把 allowed_patch_paths 留空；final gate 可将这个缺口写入 coverage_gaps。

5.4 补丁复核官

你是 V3.1 补丁复核官。你合并三个原本独立的委员会职责：diff 风险、后置验证探针、最终怀疑者。

请读取当前 diff、现有测试报告和证据卷宗，输出一份 advisory 复核。字段 diff_risks 写补丁改变了哪些可观察行为和边界。字段 post_validation 写哪些公开验证可以支持或反驳补丁；只有实际运行且有证据的失败才能触发 RETRY。字段 skeptic 写假设补丁错误时最可能的反例。字段 decision 可取 PASS 或 RETRY。RETRY 只用于有具体源码修复反馈；硬契约证据不足时，把证据缺口写进 advisory_notes，交给最终闸门和 official eval 做诊断判定。

这一步是 advisory stage。连接失败时会被降级，不会单独让整题 invalid。

补丁复核官减少了 V3 后半段的结构化调用数量。它的输出可以让求解器进行最小重试，也可以给 final gate 提供缺口说明。若它断连，fallback 输出 decision=PASS、advisory_fallback=true，最终结果会带 done_with_advisory_gap=true。

5.5 最终闸门

你是 V3.1 最终闸门。你的职责是判断当前 patch 是否可以交给 official eval。

请只根据证据卷宗、现有测试报告、补丁复核和当前 diff 边界作决定。若存在具体源码问题、测试失败、临时文件进入 diff、修改越界，请输出 CONCRETE_BLOCKER 并给出最小重试反馈。若只是 advisory stage 的网络失败或证据提示不完整，但已有 patch、路径边界清楚、没有观察到失败，请允许 PASS，并在 verification_coverage.coverage_gaps 中写清楚缺口，交由 official eval 做外部判定。

最终闸门的语义是“是否值得送评”，并非“证明已解决”。它输出 PASS 时仍可能带 coverage_gaps。watchdog 看到 done_with_advisory_gap 后把任务归入 patch_done_advisory_gap，默认标为 diagnostic_only；只有显式打开 --eval-advisory-gap 时，才把这类 patch 送 official eval。

6 相对 V3 的具体调整

位置	V3 行为	V3.1 行为
任务接入与语义盘点	多个结构化 stage 串行调用，每个 stage 都可能成为 transport failure 点	合并成一个 EvidenceDossier，失败后生成 fallback dossier，并要求求解器补读源码。

候选契约	早期生成候选契约并参与后续裁决	契约作为卷宗字段进入求解器和 final gate, 裁决后置到 patch 已存在之后。
后置验证与怀疑者	diff-risk 三路、post-validation、final-skeptic 多次结构化调用	合并为 PatchReview advisory stage, 失败时写 advisory_fallback。
final verifier	结构化失败会导致整题 infra invalid	final gate 失败时生成带 coverage_gaps 的 fallback verdict, 结果标记 done_with_advisory_gap。
checkpoint	空 diff 会删除旧 sidecar	空 diff 遇到旧非空 sidecar 时保留旧 patch, metadata 写 empty_worktree_preserved_previous_patch, 并设置 submission_eligible=false。
watchdog 分类	patch done 和内部证据完整混在一起	新增 official_eval_eligible 与 diagnostic_only, advisory gap 默认只作诊断候选。

7 实现位置

当前代码改动集中在五处。第一处是 workflows/swe_committee_v2.py, 新增 swe-committee-v3.1 workflow、V3.1 schema、中文 prompt、fallback helper、_run_v31_attempt, 并把 advisory fallback 计数写入 workflow result。第二处是 swebench/gen_prediction_workflow.py, 让 swe-committee-v3.1 默认开启 blind validation, 并复用 V3/V3.1 专用 agent prompt; 最终 patch 抽取只接受 submission_eligible=true 的 sidecar。第三处是 opencollab/opencollab/application/workflow.py, 在空 diff 和 capture failure 时显式保留上一份可校验非空 worktree sidecar, 同时把这类 sidecar 标记为不可直接提交。第四处是 scripts/swe_v3_wave_watchdog.py, 新增 patch_done_advisory_gap、official_eval_eligible、diagnostic_only、last_good_patch_* 分类字段, 并用当前 record_id+patch hash 双重绑定 prediction 与 metrics。第五处是 scripts/swe_auto_eval_driver.py, 官方评测候选按 record_id 和 patch sha 锁定, 还要反查同一 metrics 行确认 workflow eligible。

相关测试覆盖了三个方向。V3.1 happy path 检查 agent label 序列为 v31-evidence-dossier -> coder:r1 -> existing-tests-approved-validation:r1 -> v31-patch-review:r1 -> v31-final-gate:r1, 并确认非求解阶段没有写工具。transport fallback 测试模拟证据卷宗、补丁复核、最终闸门三个阶段连接失败, 结果应为 status=advisory_gap、done_with_advisory_gap=true、advisory_transport_error_count=3。ETV 失败测试确认 FAIL_WITH_EVIDENCE 会在补丁复核前回到求解器; coverage 测试确认 final gate 的 PASS 若缺少判别性 probe、原生工具审查或 API 兼容检查, 也会回到求解器。checkpoint 测试确认旧非空 sidecar 在空 diff 和 capture failure 时都被显式

保留，auto-eval 测试确认没有同 `record_id` 且同 patch hash 的 eligible metrics 时不会启动 official eval。当前目标文件 83 个 pytest 已通过。

8 科研解释

V3 效果差时，最有价值的观察并非“委员会思想失败”。真正的信号在于复杂通信协议会把基础设施抖动放大成语义失败。V3.1 的启发是：在没有足够源码事实、公开测试和 helper 审查前，契约委员会不适合直接裁决。更合理的顺序是先形成可共享证据卷宗，让求解器基于证据做最小修改，再让审查节点围绕真实 diff 做 advisory 复核。最终判断交给 official eval 和带证据的 final gate，内部缺口以可读字段保存。

因此 V3.1 更像“证据优先、裁决后置、失败可恢复”的委员会。它保留多视角讨论，但把每个视角从硬关卡改成有清楚输入输出的消息。这样能减少 transport failure 面，也能让未来做消融实验时分清模块价值：证据卷宗是否提高 patch rate，helper audit 是否降低 F2P 残留，patch review 是否减少越界修改，final gate 的 coverage gap 是否能预测 official eval。

9 下一步实验

建议用经典两题和 Pro Lite 小样本做 V3.1 smoke。第一轮只看三件事：结构化断连是否写成 `done_with_advisory_gap` 或 `infra_paused`，checkpoint 是否能恢复旧 diff，official eval 是否在 patch 产生后自动启动。第二轮再比较 V1、V3、V3.1，统计 resolved、unresolved、patch rate、workflow done rate、advisory gap rate、infra paused rate、empty patch invalid rate、token 输入输出与 official eval 一致性。

如果 V3.1 在经典两题仍然输给 V1，重点分析求解器收到的证据卷宗是否过度约束了修改空间，helper audit 是否遗漏了 V1 使用的简单路径，final gate 是否把“证据不足”反馈得太抽象。若 official eval 通过率提升，则继续做模块消融，逐步比较证据卷宗、helper audit、patch review、final gate 对成本和成功率的贡献。